

Shortest Job First (SJF)

SJF chooses the shortest ready job when the CPU becomes free. The selected job runs to completion.

1. Model

The visualizer uses one CPU, whole ticks, and CPU-only processes. **Service time** is the total CPU work needed, supplied in the input. Each execution tick reduces remaining service by one. Switching processes takes no time.

Only arrived processes are eligible. Ready processes wait for the CPU; future processes have not arrived. The queue displays insertion order, but selection searches the whole queue.

Queue entries read from left to right; [] means empty. Each timeline cell is one tick, and a dash (-) means the CPU is idle. Examples under different rules are separate scenarios.

2. Scheduling rules

Rule 1. Choose the shortest ready service time

When the CPU is free, compare the original service times of all ready processes and select the smallest. The selected process may be anywhere in the queue.

Example. B is ahead of C in the queue, but C has less service to perform.

```
Ready queue:      [B: 4 ticks] [C: 1 tick]
                  |
                  v
CPU after dispatch:      C
Ready queue left:  [B: 4 ticks]
```

Rule 2. Resolve equal lengths by arrival, then input order

For equal service times, choose the earlier arrival. If arrival times also match, choose the process listed first in the input. IDs do not determine priority.

Example. The CPU becomes free at time 4. B and C each need two ticks.

```
Case 1: different arrivals
C arrived at 1; B arrived at 2  -> choose C

Case 2: same arrival
B and C arrived at 1
Input order: [B] [C]          -> choose B
```

Rule 3. Do not interrupt a running process

SJF is non-preemptive. A shorter process arriving later joins the queue and waits until the current process finishes.

Example. A arrives at 0 with service 5. B arrives at 1 with service 1.

```

Time 0   1   2   3   4   5   6
      +---+---+---+---+---+---+
CPU   | A | A | A | A | A | B |
      +---+---+---+---+---+---+

t=1: B arrives with only 1 tick of service.
      A keeps the CPU until t=5.

```

Rule 4. Finish at zero remaining service

Remove the finished process, admit arrivals at that time, then select the shortest ready job. Dispatch takes no extra tick.

Example. A finishes at 5. B is waiting with service 4; C arrives at 5 with service 1.

```

At t=5:
1. Finish A          CPU: free   Ready: [B: 4]
2. Admit C           CPU: free   Ready: [B: 4] [C: 1]
3. Select C          CPU: C      Ready: [B: 4]

```

Rule 5. Do not wait for a future short job

If the CPU is free and a process is ready, run a ready process now. Idle only when nothing is ready. Stop when all processes finish.

Example. A arrives at 3 with service 2; B arrives at 4 with service 1.

```

Time 0   1   2   3   4   5   6
      +---+---+---+---+---+---+
CPU   | - | - | - | A | A | B |
      +---+---+---+---+---+---+

t=3: A starts; B has not arrived.
t=4: B arrives; A continues to completion.

```

3. Events at a tick boundary

After accounting for the preceding tick:

1. Remove the running process if it has finished.
2. Admit all arrivals at the current time, in input order.
3. If the CPU is free, select the shortest ready job, using Rule 2 for ties.

4. Worked example

Input order is A, B, C. All times are in ticks.

<i>Process</i>	<i>Arrival</i>	<i>Service</i>
A	0	5
B	1	2
C	1	1

```

Time 0   1   2   3   4   5   6   7   8
      +---+---+---+---+---+---+---+
CPU  | A | A | A | A | A | C | B | B |
      +---+---+---+---+---+---+---+

```

t=0: Only A is ready, so A starts.

t=1: B and C arrive; neither interrupts A.

t=5: A finishes; C (1 tick) is shorter than B (2 ticks).

t=6: C finishes; B starts.

t=8: B finishes; all processes are done.